Reply to S. Danilov's comments on the mEVP communication study

N. Koldunov, 6 August 2026

We built the variant you describe, and it changes the study's conclusion. Written as you write it — the ring computation folded into the loops that already exist rather than into kernels of its own — the wide halo removes **58% of the sea-ice cost** on CORE2 at its operating point, against 11% for our version of the same scheme, and matches the delayed exchange we had rejected. The two runs compute the same ring and put identical message and byte counts on the wire; they differ by 360 kernel launches per time step. Our earlier conclusion, that exactness costs most of the saving, was measuring our implementation.

CORE2, one node, four GPUs, all six in one allocation	model step	sea-ice cost	
every sub-cycle (reference)	0.0667 s	15.7 ms	—
delayed exchange (stale halo), K=8	−14.4%	*6.6 ms*	−58.0%
wide halo, standard form, our kernels	−3.0%	13.9 ms	−11.5%
wide halo, divergence form, our kernels	+1.1%	16.1 ms	+2.6%
wide halo, standard form, your loops	**−14.1%**	**6.8 ms**	**−56.7%**
wide halo, divergence form, your loops	**−14.4%**	**6.5 ms**	**−58.6%**

The delayed exchange is in the table because it is the scheme we rejected for putting the domain decomposition into the ice field, and because we reported it as saving about three times what the wide halo saves. **It no longer does.** In the divergence form the widened-loop wide halo matches it exactly on the model step and is 1.5% ahead of it on the ice; in the standard form it is within a third of a percent. The wide halo still computes the ring and still refreshes it exactly every window, so it still leaves no imprint. The trade we reported between speed and a clean ice field was a property of how we had written the ring, and there is now no speed argument for accepting a stale halo at all.

Giving up the owner's summation order costs what you predicted: after a time step of 120 sub-cycles the ice velocity differs from the exact wide halo by 8 to 15 units in the last place.

Point 1, the auxiliary fields. We instrumented the per-step exchange to report, field by field, the difference between the owner's arriving value and the value the receiving process already holds. Nine of the eleven arrive *exactly* equal — ice velocity, concentration, thickness, snow, ocean velocity, wind stress — because FESOM's own halo exchange has already put them there, as you said. The two right-hand sides do not: they are assembled and area-scaled over owned nodes only, so a halo node has no correct value at all. But the redundancy stops at the first ring, and rings $2 \dots K$ have no other source, so what could be removed is 37% of that exchange at $K{=}2$ and 17% at $K{=}4$. Since the exchange is 5.4% of the data the scheme moves and one message per neighbour per step, removing it would gain under one percent of the traffic and no messages. We have not implemented the trimmed version, and that is why.

Point 1, the kernel structure. This is the part we could not bound before, and it was much the larger. Our wide halo adds 362 kernel launches per model step, each on a slower launch path than the owned kernel it duplicates: the ring kernels carry enough array descriptors to cross a threshold in Kokkos above which arguments are staged through constant memory. The gap between consecutive launches is 14.3 μs for those against 4.4 μs for the owned kernels. Per step the ring kernels cost 5.9 ms of inter-kernel idle against 2.5 ms of arithmetic.

The two forms become indistinguishable. In our implementation the standard and divergence forms differed by 3.5 percentage points, and the previous note decomposed that difference into a message term and a byte term. With the ring folded into the existing loops the difference is zero to four decimals, while the divergence form still ships 40% fewer numbers. The gap we decomposed was our kernel structure. On CPU at 512 cores the picture differs and favours your form: there the ring assembly is arithmetic-bound, and folding the loops removes a recomputation the divergence form was doing about six times over, worth 3 percentage points, while in the standard form the same change costs 5.

Point 2, extending the computation over the ring. You were right, and the correction runs deeper than we allowed. Recomputing the ring does remove the imprint — averaging $|\Delta$ ice thickness$|$ on the sub-domain boundaries against the interior over 60 days on fArc gives 0.87, against 0.85 for two identical runs and 2.52 for the delayed exchange. What we wrote was that the same change removes most of the saving and that we had found no intermediate that was both cheap and clean. That was a statement about our kernels, not about the scheme.

Open. The table is CORE2 at its operating point; fArc, DARS and NG5 are queued. The 60-day boundary test is being repeated for this variant and is the gate we hold it to, since it is no longer bit-identical and cannot be checked the way the exact version was.

All numbers are from the runs described in `danilov_mevp_report.pdf`; the output and job scripts can be shared.